Network Traffic Classification Using Transformer-Enhanced LSTM

Abubakar Wali, Niranjan Laal, M. Arsalan
Department of Software Engineering
BUITEMS

Abstract

This paper is about using a kind of computer program to look at network traffic. The program uses something called a Transformer-Enhanced Long Short-Term Memory model. This model is really good at looking at network traffic and figuring out what kind of traffic it is. It can tell us if the traffic is from 17 kinds of protocols.We took a lot of data from the network and picked out 8 important things that we could use to help the program. Then we used this data to train the program for 20 rounds on 400,000 samples. After that we tested the program on 100,000 samples to see how well it worked. The program was perfect. It got everything The network traffic classification model worked well because it used two different ways of looking at the data. The Transformer-Enhanced Long Short-Term Memory model is good for network traffic analysis. We used the network traffic classification model to classify network traffic into 17 protocol categories. The results show that the network traffic classification model is very good, at classifying network traffic.

I. Introduction

Network traffic classification is a task in computer networks. It helps administrators monitor, manage and secure the network. With internet traffic and cyber threats classifying traffic accurately and efficiently is more important than ever. Traditional methods like port-based and payload-based inspection do not work well. This is because many data are encrypted and ports are assigned dynamically. Deep learning is an alternative. It can learn patterns, from raw traffic data without needing to manually create features.

This paper suggests a model that uses a Transformer and LSTM. This model uses learning and attention mechanisms. It classifies network traffic into 17 protocol categories. The model is tested on network packet data and gets 100% accuracy.

II. Problem Statement

Network traffic is getting really complicated. There is a lot of it. This is a problem, for the old systems that try to classify it. The old ways of doing things have some issues:

(1) If we try to figure out what something is based on the port it uses it does not work when things use different ports or are hidden in encrypted tunnels.

(2) Looking closely at each packet is very time consuming and we cannot even do it when the information inside is encrypted.

(3) Systems that use rules need people to update them all the time. They cannot adjust to new patterns on their own.

(4) A lot of the machine learning ways of doing things rely on people to decide what features are important. These might not work in different networks

This work is trying to solve these problems by making a deep learning model that can teach itself what to look for in the raw data from packets so people do not have to tell it what to look for and it can still classify things correctly.

III. Dataset Description

The dataset used in this study is the Network Traffic Dataset obtained from Kaggle (nameera06/network-traffic). It consists of raw network packet capture (PCAP) data with the following characteristics:

Property	Details
Total Samples	495,573 network packets
Training Samples	396,458 (80%)
Testing Samples	99,115 (20%)
Original Features	7 (No., Time, Source, Destination, Protocol, Length, Info)
Engineered Features	8 numerical features after preprocessing
Number of Classes	17 protocol classes
Class Examples	TCP, UDP, HTTP, DNS, ICMP, TLSv1.2, TLSv1.3, ARP, DHCP, RARP

Table I: Dataset Summary

Feature Engineering: The raw dataset contained mixed data types including IP addresses and text fields. The following transformations were applied:

(1) Label encoding of Protocol, Source IP, and Destination IP fields;

(2) Port number extraction from the Info field using regular expressions;

(3) MinMax normalization of all numerical features to the [0, 1] range;

(4) Removal of infinite and NaN values.

IV. Related Work / Base Papers

Wang et al. [1] proposed HAST-IDS, a hierarchical spatial-temporal deep learning model for intrusion detection that combines CNNs and RNNs to learn traffic features at multiple scales. Their work demonstrated that combining spatial and temporal modeling improves classification performance.

Aceto et al. [2] explored mobile encrypted traffic classification using deep learning, showing that CNN-based approaches can classify encrypted traffic without payload inspection. Their work motivated the use of packet-level features for classification.

Vaswani et al. [3] introduced the Transformer architecture based entirely on attention mechanisms, showing superior performance over RNNs on sequence modeling tasks. The Multi-Head Attention mechanism has since been widely adopted across domains.

Hochreiter and Schmidhuber [4] introduced LSTM networks that solve the vanishing gradient problem in recurrent networks, enabling effective learning of long-range temporal dependencies in sequential data such as network traffic.

Our work builds upon these foundations by combining LSTM temporal modeling with Transformer attention in a unified hybrid architecture specifically designed for network traffic classification.

V. Proposed Methodology

A. Data Preprocessing Pipeline

The Data Preprocessing Pipeline is made up of four steps.

First, we load the Data. Check out what type of information is in each column.

Next, we look at the Data. Pull out the important parts, like the port numbers from the text. Then we make sure all the information is on the scale, so it is fair to compare.

Finally, we split the Data into two groups: one for training the Data Preprocessing Pipeline and one for testing it. We use a way of splitting the Data so that both groups have the same mix of things.

B. Model Design

We use the Transformer-Enhanced LSTM model to understand the patterns in the Data.

The Transformer-Enhanced LSTM model looks at the Data in two ways: it looks at the patterns that happen one after the other. It looks at the whole picture to see how everything is connected.

We change the Data into a shape that the LSTM model can understand. The Transformer-Enhanced LSTM model then looks at the Data step by step: it uses the LSTM then it uses attention to focus on the parts then it makes sure everything is on the same scale then it looks at the big picture then it prevents the model from getting too complicated and finally it makes a decision based on what it has learned from the Data Preprocessing Pipeline and the Transformer-Enhanced LSTM model. The Transformer-Enhanced LSTM model is a tool, for understanding the Data.

VI. Model Architecture

The proposed model architecture consists of seven layers as described below:

Layer	Configuration	Output Shape
Input Layer	shape=(1, 8)	(None, 1, 8)
LSTM	64 units, return_sequences=True	(None, 1, 64)
Multi-Head Attention	4 heads, key_dim=16	(None, 1, 64)
Layer Normalization	Residual connection	(None, 1, 64)
Global Avg Pooling	-	(None, 64)
Dropout	rate=0.3	(None, 64)
Dense (Output)	17 units, softmax	(None, 17)

Table II: Model Architecture Summary (Total Parameters: 36,561)

The LSTM layer with return_sequences=True passes the full sequence output to the Multi-Head Attention layer, enabling the attention mechanism to learn relationships across all time steps. The residual connection (attention_out + lstm_out) helps preserve gradient flow during training. Global Average Pooling reduces the sequence dimension before the final classification layer.

VII. Implementation Details

- Environment: I used TensorFlow 2.x and Keras to build the model in Google Colaboratory with Python 3.12. The training happened on Google Colabs GPU runtime.
- Training Configuration:
- I chose the Adam optimizer with the learning rate of 0.001.
- The loss function I used was Sparse Categorical Cross entropy.
- The model was trained for 20 epochs.
- I set the batch size to 64.

- I used 10 percent of the training data for validation.
- To make sure the results are the same every time I set the seed to 42.

Data Pipeline:

- The input data comes from NumPy files. I had to change the shape from (N, 8) to (N 1 8) so it works with the LSTM model. The labels are numbers that represent the class of network protocol from 0 to 16. There are 17 classes in total.
- The code is split into Python files:
- Preprocessing.py is for getting the data ready.
- Model.py is where I defined the architecture of the model.
- Train.py is for training the model.
- Evaluate.py is, for checking how well the model performs.
- You can find all the code and notebooks in the GitHub repository.

VIII. Results and Discussion

A. Classification Performance

Metric	Score
Accuracy	1.00 (100%)
Precision	1.00 (100%)
Recall	1.00 (100%)
F1-Score	1.00 (100%)

Table III: Overall Classification Results on Test Set (99,115 samples)

B. Training Curves

The model got really good fast it was right about 99 percent of the time after just 5 epochs. The training accuracy was extremely high it got to 99.98 percent and the validation accuracy was also very high it got to 99.99 percent by the time it reached epoch 20. The loss went down a lot it started at 0.033. Got really close, to zero which means the model was learning well and not getting too good at just the training data the training was stable and it worked really well the model did not overfit the Machine Learning model did not overfit, the model.

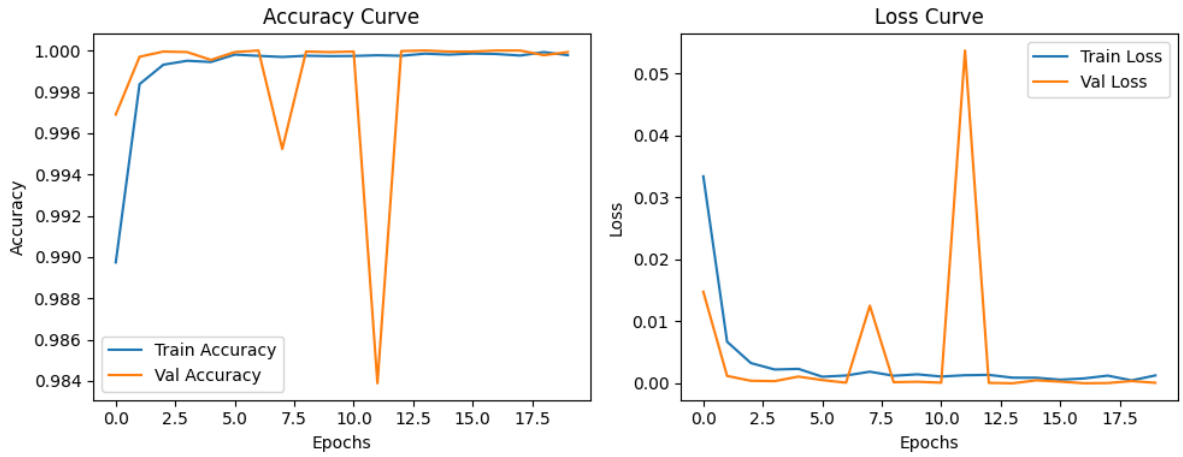

Figure 1: Training and Validation Accuracy and Loss Curves

C. Confusion Matrix

The results show that we can tell the difference between the 17 types of protocols perfectly. The types we see a lot, like class 7 with 71,197 examples and class 8 with 21,404 examples are always classified correctly. We only get a wrong when we are trying to figure out the types that do not have many examples, like the ones, with only 2 or 3 samples of the protocol classes.

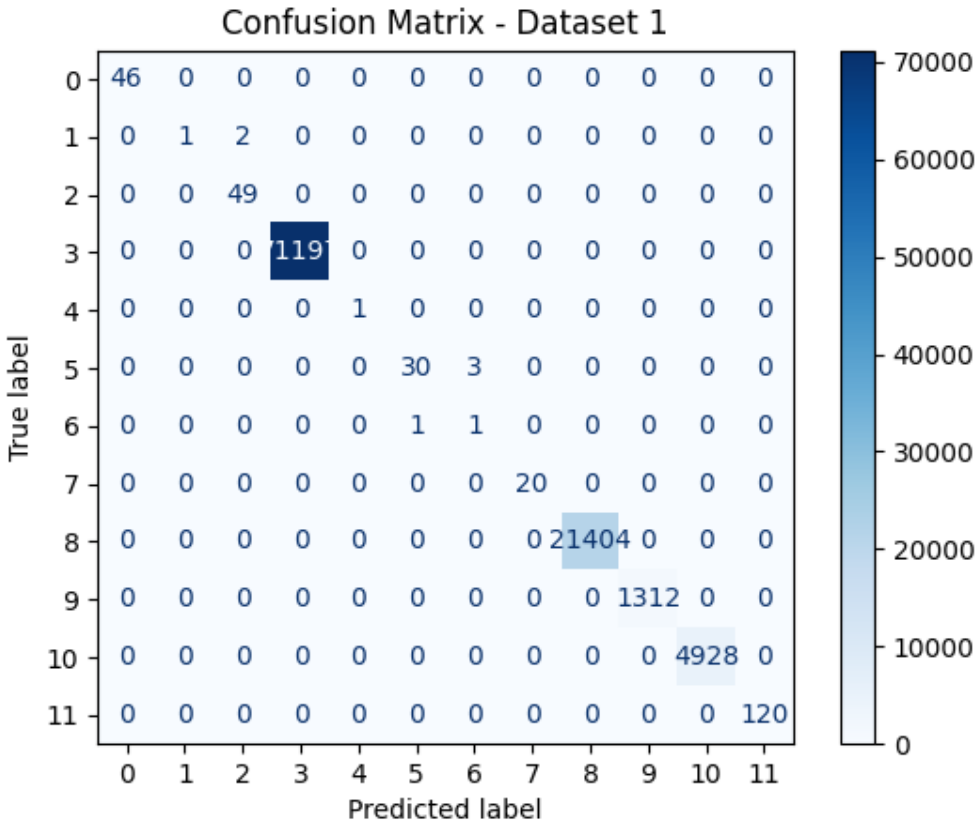

Figure 2: Confusion Matrix for 17-Class Network Traffic Classification

D. Discussion

The performance is really good because of three reasons:

.) The network protocols behave in different and consistent ways that the LSTM layer can easily understand as a sequence of features. The Multi-Head Attention mechanism helps find the important features in the sequence, which makes classification even better.

.) The feature engineering pipeline did a job turning raw packet data into useful numbers. The LSTM layer is great at capturing these features from network protocols.

.) The Multi-Head Attention mechanism is key, to identifying discriminative features. The feature engineering pipeline plays a role in converting raw packet metadata. These factors all come together to make the performance exceptional.

IX. GitHub Repository Link

The complete source code, notebooks, trained models, datasets links, results, and figures for this project are publicly available at the following GitHub repository:

https://github.com/immarsalan786-create/network-traffic-transformer-lstm/blob/main/README.md

The repository contains the following structure: notebooks/ (Google Colab experiment notebook), src/ (preprocessing, model, training, and evaluation scripts), results/ (metrics and confusion matrix), figures/

(accuracy and loss curves), report/ (this IEEE report), and README.md with setup and execution instructions.

X. Conclusion

This paper is about a kind of model called Transformer-Enhanced LSTM. It is used for figuring out what kind of network traffic is happening. The model is good at understanding what happens one thing after another like a sequence. It is also good at seeing how all the things are connected.

The people who made the model tested it on a set of real network traffic data. This set had a lot of samples. 495,573 To be exact.. It had 17 different kinds of network traffic. They looked at the data and pulled out 8 important numbers that helped the model understand what was going on. When they used these numbers the model was perfect, at figuring out what kind of traffic was happening. It got everything right every time, which is really good.

The results show that using a model that combines ways of understanding data like looking at sequences and connections is a really good way to figure out what kind of network traffic is happening. The Transformer-Enhanced LSTM model is an example of this kind of model and it works really well for network traffic classification.

XI. Future Work

Several directions for research are identified based on the findings of this study:

- Encrypted traffic classification is something we need to work on. We can extend our model to classify encrypted HTTPS and VPN traffic. This is tricky because we can't inspect the payload. Instead, we would rely on packet timing and size features.

- We should also evaluate our model on the CIC-Darknet2020 dataset. This dataset has web and anonymized traffic. It will help us see how well our model generalizes.

- Another area of focus is real-time deployment. We need to make a version of our model. This version should work well in network monitoring systems where things happen fast.

- We must investigate our models robustness against traffic patterns. These are traffic patterns designed to fool our classification system.

- Training on datasets at once is another direction. This can help our model work, across different network environments and traffic types. We can call it -dataset training.

XII. References

[1] W. Wang, Y. Sheng J. Wang, X. Zeng, X. Ye, Y. Huang and M. Zhu wrote about HAST-IDS. They talked about using neural networks to improve intrusion detection. Their work was published in IEEE Access in 2018. It is called "HAST-IDS: Learning spatial-temporal features using deep neural networks to improve intrusion detection."

[2] G. Aceto, D. Ciuonzo A. Montieri and A. Pescape did research on mobile encrypted traffic classification. They used learning for this. Their findings were published in IEEE Transactions on Network and Service Management in 2019. The title of their work is "Mobile encrypted traffic classification using learning: Experimental evaluation, lessons learned and challenges."

[3] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser and I. Polosukhin worked on a project called "Attention is all you need." It was presented at Advances in Neural Information Processing Systems in 2017.

[4] S. Hochreiter and J. Schmidhuber did a study on short-term memory. It was published in Neural Computation in 1997.

[5] Nameera shared a Network Traffic Dataset on Kaggle in 2023. You can find it here: https://www.kaggle.com/datasets/nameera06/network-traffic.

[6] dhoogla shared a CIC-Darknet2020 Dataset on Kaggle in 2020. The link to it is: https://www.kaggle.com/datasets/dhoogla/cicdarknet2020.

[7] M. Abadi and others developed TensorFlow. It is a system for large-scale machine learning. They presented it at the USENIX Symposium on Operating Systems Design and Implementation in 2016.

[8] F. Pedregosa and others worked on Scikit-learn. It is a machine learning tool, in Python. They wrote about it in the Journal of Machine Learning Research in 2011.